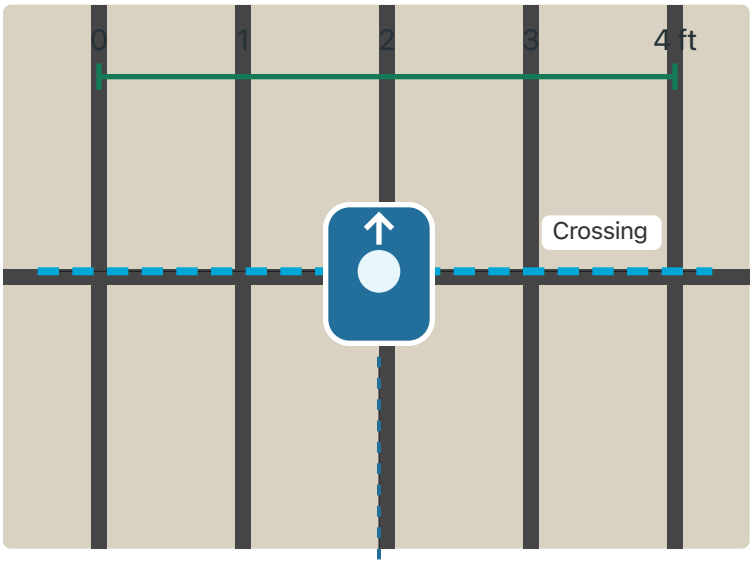
My roomberry grid runner iOS app is successfully tracking a grout line in my kitchen tiles for about 5m. The intent was to track Roomba odometry to real world distances. Now I would like to add the recognition of perpendicular grout lines occurring every tile (1 foot each) to the iOS app. What should be the approach?

Excellent progress! If GridRunner can follow a longitudinal grout line for 5 meters, you've already solved a substantial part of the problem: keeping a stable visual reference while Rosie moves.
The next step is to turn GridRunner into a **visual distance encoder**, using the perpendicular grout lines as regularly spaced calibration marks.
I'd approach this as a relatively small extension to your existing line follower, rather than introducing a completely new computer-vision system.

1. The basic idea: count tile crossings



Longitudinal grout line

Conceptual top-down view: the blue path follows the longitudinal grout line, while each transverse intersection provides a known 1-foot distance increment. Diagram not to scale.

Your kitchen floor effectively becomes a measuring ruler:

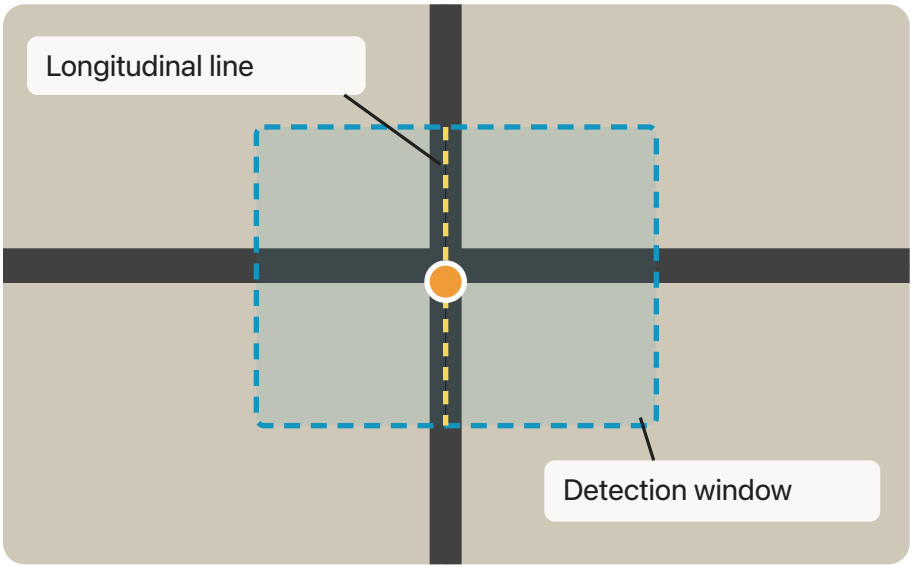
- The longitudinal grout line provides lateral alignment.
- Each perpendicular grout line is a distance landmark.
- Consecutive intersections are nominally 304.8 mm apart.
- Wheel odometry between intersections can be compared against that known distance.

The important distinction is that **we want to detect a crossing event**, not merely recognize a perpendicular line somewhere in the camera image. That makes the task much simpler and more robust.

2. Detect transverse grout lines in a small region

Since GridRunner already identifies the longitudinal line, we can reuse its geometry.

I'd add a small *crossing detector* operating on a region of interest (ROI) around the tracked line.



Camera-frame concept: look for dark horizontal grout across a narrow region centered on the existing tracked line.

A straightforward detection pipeline:

1. Convert the camera image to grayscale, as GridRunner probably already does.

2. Use the tracked longitudinal line to position a rectangular ROI.

3. For each image row within the ROI, calculate the average darkness across the ROI width.

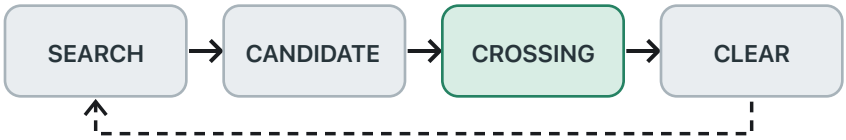
4. Look for a strong dark band spanning both sides of the longitudinal grout.

5. Confirm the band persists across a few consecutive frames.

One subtlety: the longitudinal grout itself is dark, so averaging across the entire ROI could dilute the crossing signal. Better to sample two strips, one on either side of the longitudinal line, and look for a simultaneous darkness peak.
This avoids mistaking the existing line for a perpendicular crossing.

3. Use a crossing state machine

A grout line may remain visible for dozens of video frames. We must count it exactly once.
I'd implement four states:



Count only when a confirmed intersection passes the reference line; re-arm after it clears.

For each crossing, record:

- Crossing number and timestamp.

- Cumulative wheel odometry from Rosie.

- Detection confidence.

- Position of the crossing within the camera image.
- Direction of travel.

Use a fixed reference row in the camera image. Detect when the center of the transverse grout band passes that row, rather than counting as soon as the band appears.
That gives a repeatable event location.

4. Compare wheel odometry with actual distance

This is where the experiment becomes especially useful.
Suppose Rosie's wheel encoder reports the following:

Tile crossing	Actual distance	Wheel odometry
0	0 mm	0 mm
1	304.8 mm	312 mm
2	609.6 mm	625 mm
3	914.4 mm	939 mm
4	1219.2 mm	1251 mm

Illustrative data, not measurements from Rosie.

Here the wheel odometry is overestimating distance by approximately 2.6%.
More importantly, measuring each individual tile interval can reveal wheel slip, inconsistent speed, or turning errors that a single end-to-end measurement would conceal.
A useful CSV record might look like:

```
timestamp,crossing,tile_mm,wheel_mm,confidence
12.500,0,0.0,0.0,0.98
15.210,1,304.8,312.0,0.96
17.930,2,609.6,625.0,0.99
```

The NavLink timestamps and odometry should ideally use a shared monotonic time reference or an estimated clock offset. Otherwise network latency can introduce apparent distance errors.

5. One important geometric detail

Your camera is mounted toward Rosie's rear, and its optical axis is offset about 13 mm from the Roomba centerline.

For longitudinal tracking, that offset matters. For transverse crossings, the more important measurement is the **fore-aft distance between the camera's reference point and the wheel axle**.

Fortunately, if Rosie travels straight, this fixed offset cancels when calculating distances between consecutive crossings.

However, if Rosie turns while crossing a line, the camera and wheel axle follow different trajectories. That's another reason to retain the existing straight-line controller.

Also, perspective matters. The grout line will move nonlinearly through the camera image. We don't need to solve that immediately if we're only counting crossings at one fixed reference row.

6. Suggested implementation sequence

I'd keep this incremental:

1. **Detection only:** Display transverse grout candidates as colored overlays without counting.

2. **Crossing counter:** Add the state machine and display the running count.

3. **Event logging:** Record crossing timestamps and image positions.

4. **NavLink integration:** Associate crossings with Rosie's encoder odometry.

5. **Calibration report:** Calculate per-tile odometry errors and cumulative drift.

For the first iteration, I would avoid machine learning, Hough transforms, and full perspective rectification.

Apple's Vision framework does provide contour detection through `VNDetectContoursRequest`



Apple Developer Documentation, but a custom grayscale intensity detector is likely simpler, faster, and easier to debug for your highly structured kitchen floor.

7. An improvement worth considering later

Once GridRunner reliably detects crossings, it can also track the transverse line's position *between* its first appearance and its passage through the reference row.
That provides a continuous visual motion signal between tiles, rather than only one measurement every foot.
You could then compare three independent measurements:

- Wheel odometry from Rosie.
- Continuous image displacement from GridRunner.
- Absolute tile-crossing landmarks every 304.8 mm.

That would be a particularly interesting way to quantify wheel slip.

One question before implementing this: Does GridRunner currently track the longitudinal grout line using a grayscale intensity profile, edge detection, or some other method?
I'd prefer to build the crossing detector directly on top of the existing image-processing pipeline rather than introduce a second one.

❓

A little gotcha is the led light corona in reflections of the floor right in the middle of the screen. Tracking grout-to-grout distances from the same camera perspective should provide valid distances, right?

Yes! Tracking successive perpendicular grout lines from the same camera perspective is a very good way to measure Rosie's actual travel distance. And your screenshot suggests we can do it without much additional image processing.
There's one important distinction, though:

- **Counting grout crossings:** Each crossing represents 304.8 mm of travel, regardless of camera perspective, provided Rosie travels straight and crosses the lines perpendicularly.

- **Measuring pixel displacement:** Perspective must be accounted for if you want to convert the movement of a grout line across the image into millimeters.

What I notice in your screenshot

Your actual camera preview shows a remarkably clear horizontal grout line, despite the uneven tile texture.
The bright LED reflection is concentrated near the middle, while the grout line spans almost the entire image width. That gives us plenty of usable pixels on either side of the reflection.
I would **not try to remove the LED corona** initially. Instead, use two narrow vertical sampling strips, one left and one right of the reflection.
For each video frame:

1. Calculate the average grayscale intensity of each image row in both strips.

2. Find horizontal dark bands appearing at approximately the same vertical position in both strips.

3. Reject isolated dark spots and tile texture.

4. Track the detected band as it moves through successive frames.

5. Register a crossing when it passes a fixed image row.

The existing longitudinal tracking can continue independently.

Why the fixed image row matters

Imagine that the camera sees a transverse grout line approaching from the top of the image.

The line moves toward the bottom as Rosie travels. We record the instant its center passes a designated reference row.

When the next grout line passes that **same row**, Rosie has traveled one tile length.

The camera does not need to know the absolute distance to that row. It only needs to maintain the same position and orientation relative to the robot.

Mathematically:

$$
\begin{gathered}
D \\
= \\
N \\
\times \\
304.8 \\
\text{mm}
\end{gathered}
$$

$$D = N \times 304.8 \text{ mm}$$

where

N

N is the number of complete tile intervals.

Your configured lens position of **–200 mm forward** is interesting: the camera is 200 mm behind Rosie's reference point. But that offset cancels when comparing two successive crossings at the same image row. Similarly, the 26 mm lateral offset does not affect the distance between parallel grout lines during straight travel.

A useful refinement: use the grout line itself as a moving ruler

Rather than only counting crossings, you can estimate the distance traveled *between* crossings.

Suppose a line is detected at image row 80, then 100, then 125. With a calibrated camera geometry, those positions correspond to different physical distances ahead of or behind the camera.

For a downward-facing camera over a flat floor, the mapping can be approximately linear after correcting lens distortion and perspective. If the camera is tilted, a projective mapping is needed.
You already have a **lens height setting of 85 mm**, which is useful for that calibration.
However, I would start with crossing counts, because they provide reliable ground-truth checkpoints against which to test any continuous pixel-based odometry.

One caution: the LED reflection

The reflection could affect the crossing detector in two ways:

- A dark grout line might become less distinct as it passes through the bright spot.

- Automatic exposure might change as the camera moves across tiles.

I'd use the median or a trimmed mean of the two side-strip intensity measurements, rather than rely on the center of the image. Locking camera exposure and white balance during the run would also help.

My suggested next experiment

Before modifying Rosie's steering behavior, add a diagnostic overlay showing the detected transverse line and a fixed crossing reference row.
Have GridRunner log:

```
12.410  CROSSING 0  confidence=0.97
18.522  CROSSING 1  confidence=0.99
24.630  CROSSING 2  confidence=0.96
30.745  CROSSING 3  confidence=0.98
```

At your current speed of **50 mm/s**, one 304.8 mm tile should take approximately **6.10 seconds**. That's an excellent sanity check for the detector and can help reject duplicate crossings.
At the end of a 5-meter run, you should see roughly 16 tile intervals.
One additional thought: Since your camera is looking almost straight down, we may eventually be able to extract continuous floor-relative velocity from the motion of transverse grout lines, calibrated against their known 304.8 mm spacing. That would give Rosie an independent visual odometer, not just a tile counter.

I'd implement the crossing detector first and use it to establish how accurate Rosie's wheel odometry actually is.

x